Pimpri Chinchwad Education Trust's
Legacy Of Nurturing Excellence Since 1990
PIMPRI CHINCHWAD UNIVERSITY
(Established under Govt. of Maharashtra Act No.V of 2023 and as per amended Act no. VIII of 2024)
PCET's Pimpri Chinchwad University, Pune
Learn | Grow | Achieve

PYTHON PROGRAMMING

(MINI PROJECT REPORT)

Project Title

VOWEL & CONSONANT COUNTER

Topic 5 - Sets & Strings | Project 3

Submitted by

Parag Yewale - Roll Number: 12

Om Supe - Roll Number: 11

Class / Semester: SY-C / III

Submitted to

ASHISH MISHRA

Academic Year: 2026-2027

TABLE OF CONTENTS	Page No.
1. INTRODUCTION	2
2. PROBLEM STATEMENT	2
3. OBJECTIVES	3
4. METHODOLOGY	3
5. PYTHON CONCEPTS USED	4
6. SOURCE CODE AND IMPLEMENTATION	5
7. PROGRAM EXECUTION AND OUTPUT	7
8. CONCLUSION	9
9. GITHUB REPOSITORY	9

1. INTRODUCTION

The Vowel & Consonant Counter is a Python mini project that analyzes a word or sentence and reports the number of vowels and consonants. It also displays the total number of English letters, the number of ignored characters, and the distinct letters found in each category.

The project applies the Sets & Strings topic from the Python Programming course. Strings hold the entered text, while sets support membership checks and store unique letters. A simple menu lets the user analyze several inputs before choosing to exit.

2. PROBLEM STATEMENT

Manually counting vowels and consonants becomes slow and error-prone when text contains repeated letters, mixed letter case, spaces, digits or punctuation. A program is needed to classify each English letter consistently and present the results clearly.

The program counts a, e, i, o and u as vowels and the other English letters, including y, as consonants. Uppercase and lowercase forms are equivalent. All other characters are ignored. Each occurrence contributes to the count, while the unique-letter lists contain each letter only once.

3. OBJECTIVES

1. Accept a word or sentence through a menu-driven console program.

2. Count English vowels and consonants correctly, regardless of letter case.

3. Exclude spaces, digits, punctuation and non-English characters from letter counts.

4. Use sets to identify vowels and record distinct vowels and consonants.

5. Display clear results and handle invalid menu choices and blank input.

6. Test the program and document its implementation, output and GitHub location.

4. METHODOLOGY

1. Define the input rules: process English letters A-Z and a-z; treat a, e, i, o and u as vowels; ignore other characters.

2. Create a vowel set and an English-letter set. Initialize two integer counters and two empty sets for unique letters.

3. Read the user's menu choice. For analysis, accept a string and reject empty or whitespace-only input.

4. Visit each character. Skip it if it is outside the English-letter set; otherwise convert it to lowercase and check its membership in the vowel set.

5. Increment the appropriate counter and add the lowercase letter to the corresponding unique-letter set.

6. Return the results in a dictionary. Sort the unique sets for display and show total letters and ignored characters.

7. Repeat the menu until Exit is selected. Handle end-of-input and Ctrl+C cleanly.

8. Run automated tests, capture execution output, and upload the complete project and report to GitHub.

Algorithm flow: Start - Display menu - Read choice - Validate input - Scan characters - Classify and count - Display results - Return to menu - Exit.

5. PYTHON CONCEPTS USED

5.1 Variables and Data Types

The input is a string. Integer variables store the counts of vowels and consonants. These values are updated as the text is scanned.

5.2 Strings and String Methods

input() reads a line of text. lower() converts an English letter to lowercase so that A and a are classified alike. strip() checks menu choices and detects blank input without removing spaces from the analyzed text.

5.3 Sets and Membership

VOWELS and LETTERS are sets. The in and not in operators check membership. add() records each distinct lowercase letter only once. Repeated letters still increase the occurrence counters.

5.4 Conditional Statements

if, elif and else select menu actions, validate input, and choose whether a valid English letter is a vowel or consonant.

5.5 Loops and Loop Control

A for loop scans the input. A while loop repeats the menu. continue skips ignored characters or restarts the menu after blank input; break ends the program.

5.6 Functions and Dictionaries

count_letters() performs analysis, display_result() prints results, and main() manages interaction. A dictionary returns named counts and unique-letter sets, keeping the logic easy to test.

5.7 Sorting and Output

sorted() places the unique letters in alphabetical order. join() formats them as a readable list. print() shows the menu and counts.

5.8 Exception Handling and Testing

try and except handle EOFError and KeyboardInterrupt. The standard unittest module tests counting rules, menu validation and exit behavior without installing external packages.

6. SOURCE CODE AND IMPLEMENTATION

File: main.py. The following listing is the complete application source code.

```
"""Python mini project: Vowel & Consonant Counter (Sets & Strings).

Students: Parag Yewale (12) and Om Supe (11).
"""

VOWELS = set("aeiou")
LETTERS = set("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ")


def count_letters(text):
    """Count English letters; ignore all other characters.

    Repeated letters count separately. Unique letters are also returned.
    The letter y is treated as a consonant.
    """
    vowels = 0
    consonants = 0
    unique_vowels = set()
    unique_consonants = set()

    for character in text:
        if character not in LETTERS:
            continue
        letter = character.lower()
        if letter in VOWELS:
            vowels += 1
            unique_vowels.add(letter)
        else:
            consonants += 1
            unique_consonants.add(letter)

    return {
        "vowels": vowels,
        "consonants": consonants,
        "ignored": len(text) - vowels - consonants,
        "unique_vowels": unique_vowels,
        "unique_consonants": unique_consonants,
    }
```

6. SOURCE CODE AND IMPLEMENTATION CONTINUED

```
def display_result(text, result):
    """Display counts and alphabetically sorted unique letters."""
    print("\n--- ANALYSIS RESULT ---")
    print("Input text        :", text)
    print("Vowels            :", result["vowels"])
    print("Consonants        :", result["consonants"])
    print("Total letters     :", result["vowels"] + result["consonants"])
    print("Ignored characters:", result["ignored"])
    for label, key in [("Unique vowels     :", "unique_vowels"),
                       ("Unique consonants :", "unique_consonants")]:
        letters = ", ".join(sorted(result[key])) or "None"
        print(label, letters)


def main():
    """Run the menu until the user selects Exit."""
    while True:
        print("\n===== VOWEL & CONSONANT COUNTER =====")
        print("1. Analyze text")
        print("2. Exit")
        try:
            choice = input("Enter your choice (1-2): ").strip()
            if choice == "1":
                text = input("Enter a word or sentence: ")
                if not text.strip():
                    print("Please enter at least one non-space character.")
                    continue
                display_result(text, count_letters(text))
            elif choice == "2":
                print("Thank you for using the counter!")
                break
            else:
                print("Invalid choice! Please enter 1 or 2.")
        except (EOFError, KeyboardInterrupt):
            print("\nCounter closed.")
            break


if __name__ == "__main__":
    main()
```

Implementation note: The English-letter membership check occurs before lowercase conversion. This keeps the stated A-Z and a-z scope consistent for Unicode input. The two unique-letter sets hold at most 26 different letters.

For n input characters, analysis takes O(n) time and uses a fixed amount of additional space for the counters and English-letter sets, excluding the input string itself.

7. PROGRAM EXECUTION AND OUTPUT

Software requirement: Python 3. The application uses only the Python standard library. Open a terminal in the project folder and run `python main.py`. Choose 1 to analyze text and 2 to exit.

The following captures display output recorded from actual executions of main.py.

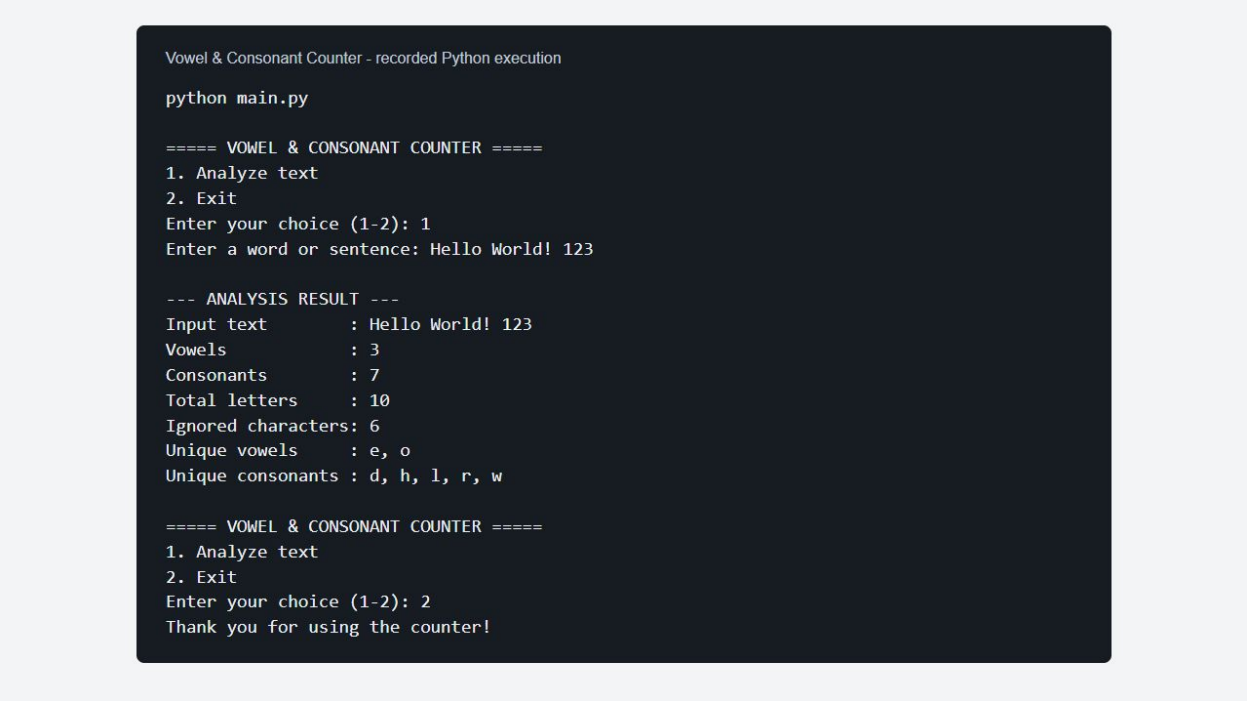


Figure 1 Mixed sentence containing uppercase and lowercase letters, spaces, punctuation and digits

For Hello World! 123, the program reports 3 vowels, 7 consonants and 6 ignored characters. Repeated occurrences of l count separately, while l appears only once in the unique-consonant list.

Automated validation: All 12 tests in test_main.py passed. Run them with `python -m unittest -v`.

Test input	Vowels	Consonants	Ignored
Hello World! 123	3	7	6
AEIOUaeiou	10	0	0
Rhythm	0	6	0
123 !?	0	0	6
AaBbAa	4	2	0

7. PROGRAM EXECUTION AND OUTPUT CONTINUED

The menu rejects an invalid choice and an input containing only spaces, then successfully analyzes a new string. Uppercase and lowercase vowels are both counted.

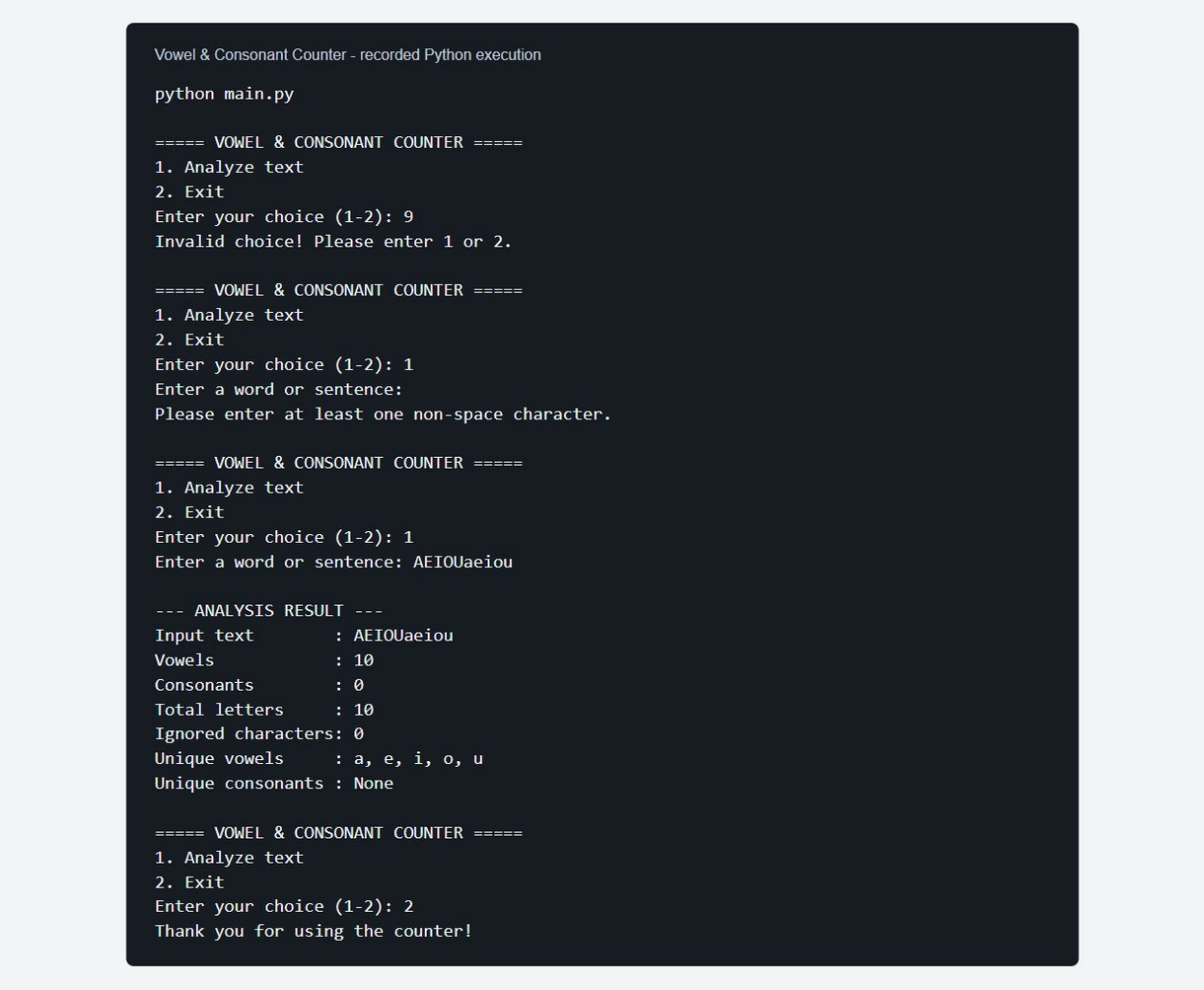


Figure 2 Invalid menu choice, blank-input validation, vowel-only input and exit

The remaining tests verify the full alphabet, empty strings and whitespace at the analysis-function level, non-English characters, repeated-letter sets, and clean handling of end-of-input and keyboard interruption. A blank console input is rejected, while the reusable counting function returns zero counts for an empty string.

8. CONCLUSION

The Vowel & Consonant Counter implements the selected Sets & Strings mini project using a simple Python console interface. It counts English vowels and consonants, handles letter case consistently, and distinguishes occurrence counts from unique-letter sets.

The implementation demonstrates string processing, set membership, variables, dictionaries, loops, conditional statements, functions, input and output, and basic exception handling. The test results and recorded executions verify the expected behavior for ordinary text and edge cases.

The program is limited to English letters A-Z and a-z. Accented letters, other scripts, emoji, digits, spaces and punctuation are counted as ignored characters. This explicit scope keeps the classification rules clear. A future version could add file input, saved results or a graphical interface.

9. GITHUB REPOSITORY

The repository contains the complete source code, automated tests, recorded sample output, execution captures and this project report.

Item	Details
GitHub username	omsupe28
Repository name	vowel-consonant-counter
Application	main.py
Test suite	test_main.py - 12 passing tests

Repository URL

https://github.com/omsupe28/vowel-consonant-counter

Student details

Student	Roll Number
Parag Yewale	12
Om Supe	11